PROJECT REPORT

Capstone_2 - LLMOPS(Agentic RAG ChatDoc)

Technology Stack:

- Python
- LangGraph
- LangChain
- Azure OpenAI
- FAISS
- FastAPI
- Streamlit
- GitHub Actions

Agentic RAG ChatDoctor: Multi-Agent Healthcare Knowledge Assistant using LangGraph, Azure OpenAI, FAISS, FastAPI, and CI/CD.

1. Executive Summary

The healthcare industry requires quick and reliable access to medical information. Traditional chatbots often generate responses without grounding them in trusted sources, leading to hallucinations and inaccurate recommendations.

This project presents **Agentic RAG ChatDoctor**, a Healthcare Knowledge Assistant that combines Retrieval-Augmented Generation (RAG) with a multi-agent orchestration framework. The system retrieves relevant medical information from a healthcare knowledge base, validates retrieved information, and generates context-aware responses using Azure OpenAI models.

The solution utilizes LangGraph to orchestrate multiple agents, FAISS as the vector database for semantic search, FastAPI for backend deployment, and Streamlit for user interaction.

2. Problem Statement

Healthcare users frequently seek information regarding diseases, symptoms, medications, and treatment options. Large Language Models may provide incorrect or hallucinated responses if they rely solely on parametric knowledge.

The objective of this project is to build a healthcare chatbot capable of:

- Retrieving relevant medical information from a knowledge base.
- Generating evidence-grounded responses.
- Supporting agent-driven reasoning.
- Validating retrieval quality before answer generation.
- Providing source-backed medical information.
- Deploying as a scalable API and chat application.

This aligns with the Healthcare Knowledge Assistant use case specified in the capstone requirements.

Expected Outcomes

By implementing the Agentic RAG architecture, the system is expected to:

- Improve answer accuracy and relevance.
- Minimize hallucinated responses.
- Enhance retrieval effectiveness.
- Provide explainable and source-grounded outputs.
- Deliver a better user experience compared to traditional chatbot systems.
- Demonstrate the effectiveness of multi-agent workflows in domain-specific question-answering applications.

This project showcases the practical application of Generative AI, Retrieval-Augmented Generation, Agentic AI, and modern deployment practices in building a scalable and reliable healthcare knowledge assistant.

The scope of the project includes:

- Processing healthcare-related datasets.

- Generating vector embeddings for semantic search.
- Building a FAISS-based knowledge repository.
- Implementing baseline RAG and Agentic RAG architectures.
- Developing specialized agents for retrieval, validation, and tool execution.
- Supporting healthcare question-answering functionality.
- Providing an easily deployable and scalable architecture.

3. System Architecture

- Dataset Description
- Data Ingestion
- Text Chunking
- Embedding Generation
- FAISS Vector Database
- RAG Pipeline
- LangGraph Workflow
- Prompt Engineering
- FastAPI Integration
- Streamlit UI
- CI/CD Pipeline
- Evaluation
- Challenges
- Conclusion

4. Data Collection and Dataset Preparation

4.1 Overview

The foundation of any Retrieval-Augmented Generation (RAG) system lies in the quality and diversity of the dataset used for knowledge retrieval. Since this project aims to develop an intelligent healthcare assistant capable of answering medical queries, multiple healthcare-related datasets were collected and integrated to build a comprehensive medical knowledge base.

Instead of relying on a single dataset, three different medical datasets were utilized to improve domain coverage and provide information related to diseases, symptoms, causes, precautions,

medications, and treatment recommendations. Combining multiple datasets enhanced the knowledge repository and increased the likelihood of retrieving relevant information during user interactions.

The datasets are stored within the project's **data** directory and serve as the primary source of medical knowledge for the Agentic RAG pipeline.

4.2 Dataset Files Used

The following datasets were used in the project:

Dataset Name	Purpose
healthcare_disease_dataset.csv	General healthcare disease information
medical_dataset.csv	Disease descriptions, precautions, medications, and treatments
Symptom2Disease.csv	Symptom-to-disease mapping for diagnosis-related retrieval

4.3 Healthcare Disease Dataset

Objective

The healthcare disease dataset was incorporated to provide detailed information regarding common diseases, their symptoms, causes, risk factors, and treatment recommendations.

Importance

Many healthcare queries submitted by users revolve around understanding a disease, its causes, symptoms, and possible preventive measures. The healthcare disease dataset acts as a foundational medical reference source that enables the chatbot to retrieve disease-specific information.

Typical Information Available

Information Type
Disease Name
Symptoms
Causes

Risk Factors
Prevention
Treatment

4.4 Medical Dataset

Objective

The medical dataset serves as the primary healthcare knowledge source containing structured information related to diseases and their associated medical details.

Role in Project

This dataset enhances the chatbot's ability to provide:

- Disease descriptions
- Medical precautions
- Recommended medications
- Lifestyle recommendations
- Treatment-related guidance

Information Categories

Category	Description
Disease	Name of condition
Description	Details about disease
Precautions	Safety measures
Medications	Common medications
Diet	Dietary suggestions
Treatment	General treatment recommendations

Benefits

The inclusion of this dataset improves answer completeness by allowing the chatbot to provide more detailed healthcare guidance rather than merely identifying diseases.

Example

For a user query:

Plain Text
How can diabetes be managed?

The retrieved context may include:

- Disease description
- Dietary recommendations
- Lifestyle changes
- Medication information

This enables the generated response to be comprehensive and informative.

4.5 Symptom2Disease Dataset

Objective

The Symptom2Disease dataset establishes relationships between symptoms and potential diseases.

Importance

Many users do not know the name of a disease and instead describe their symptoms. The chatbot must therefore understand symptom patterns and retrieve relevant disease information.

This dataset enables symptom-centered retrieval.

Example User Queries

Plain Text
I have fever, cough, and headache.

What disease causes chest pain and difficulty breathing?

What could constant fatigue indicate?

The Symptom2Disease dataset helps map symptoms to potential medical conditions, allowing the system to retrieve the most relevant healthcare information.

Typical Structure

Symptoms	Disease
Fever, cough	Flu
Increased thirst, fatigue	Diabetes
Shortness of breath	Asthma

4.6 Why Multiple Datasets Were Used

Using a single healthcare dataset often limits the coverage of medical knowledge. Different datasets contain different types of healthcare information.

The integration of multiple datasets offers several advantages:

Benefit	Explanation
Higher Knowledge Coverage	Covers a larger number of diseases and symptoms
Improved Retrieval Accuracy	More relevant context retrieval
Better Response Quality	Richer medical information
Reduced Hallucination	Increased grounding through factual data
Enhanced User Experience	More comprehensive answers

5. Data Loading Using DirectoryLoader

5.1 Overview

After collecting the healthcare datasets, the next step involved loading the data into the Agentic RAG pipeline. Instead of loading each file individually, the project utilizes LangChain's **DirectoryLoader** to automatically discover and load healthcare datasets from a specified directory.

Using DirectoryLoader simplifies dataset management by allowing the system to process multiple files from a single location. This approach improves scalability and makes it easier to extend the

knowledge base by simply adding new datasets to the data folder without modifying the application code.

```
#2 Load The DataSet
csv_loader = DirectoryLoader("./data", glob="*.csv", loader_cls=CSVLoader, loader_kwargs={"c

medical_docs = csv_loader.load()
```

6. Text Chunking Using RecursiveCharacterTextSplitter

6.1 Overview

After loading the healthcare datasets, the documents were too large to be directly processed by the embedding model and retrieval system. Large documents can reduce retrieval accuracy because relevant information may be buried within lengthy text blocks. To address this challenge, the project implements **RecursiveCharacterTextSplitter**, a LangChain text-splitting technique that divides large documents into smaller, semantically meaningful chunks.

Text chunking is a critical step in the Retrieval-Augmented Generation (RAG) pipeline because it significantly influences retrieval quality, embedding effectiveness, and the relevance of generated responses.

6.2 Configuration

The following configuration was used for chunk generation:

Parameter	Value
Chunk Size	1000
Chunk Overlap	200
Splitter Type	RecursiveCharacterTextSplitter

7. Embedding Generation Using Azure OpenAI Embeddings

7.1 Overview

After generating text chunks, the next step involved converting textual information into numerical vector representations using Azure OpenAI Embeddings. Since vector databases cannot directly process raw text, each chunk must be transformed into a high-dimensional embedding that captures its semantic meaning.

Embeddings enable the system to perform similarity searches based on meaning rather than exact keyword matching, allowing the chatbot to retrieve relevant healthcare information even when users phrase questions differently.

7.2 Embedding Model Used

The project utilizes Azure OpenAI Embeddings.

Parameter	Value
Provider	Azure OpenAI
Purpose	Semantic Embedding Generation
Input	Text Chunks
Output	Numerical Vector Embeddings

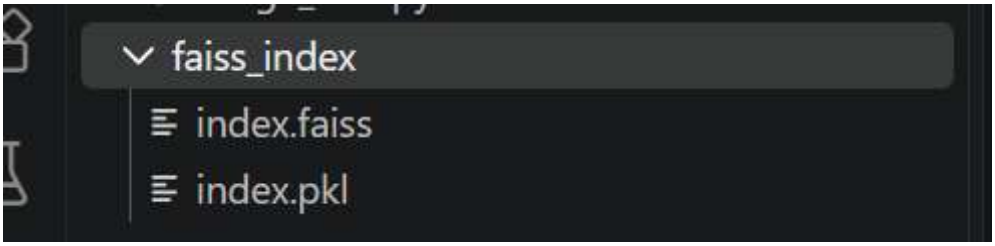
8. Vector Store Creation Using FAISS (Batch Processing)

8.1 Overview

After generating embeddings for all healthcare document chunks, the next step is to store these vector representations in a vector database. Since the healthcare datasets contain a large number of document chunks, generating and storing embeddings simultaneously can consume significant memory and computational resources. To overcome this challenge, the project implements **batch-**

wise vector store creation, where embeddings are processed and indexed in smaller groups before being combined into a single FAISS vector database.

This approach improves scalability, memory efficiency, and processing speed while ensuring that the complete healthcare knowledge base is successfully indexed.



File Description

File	Purpose
index.faiss	Stores vector embeddings
index.pkl	Stores metadata and mappings

This storage structure allows the application to reload the vector database without regenerating embeddings.

9. Hybrid Retrieval System Using Semantic Retriever and BM25 Retriever

9.1 Overview

After creating the FAISS vector database, the next step in the Agentic RAG ChatDoctor system was implementing a **Hybrid Retrieval Architecture** using both **Semantic Retrieval (FAISS Retriever)** and **BM25 Retriever**.

In healthcare applications, relying solely on vector search or keyword search can sometimes lead to incomplete retrieval results. Semantic retrieval excels at understanding meaning and context, while BM25 excels at matching exact keywords, disease names, medication names, and medical terminologies.

To leverage the strengths of both approaches, a hybrid retrieval mechanism was implemented that combines semantic relevance with keyword-based relevance, thereby improving the quality of retrieved medical information.

9.2 Objective

The objectives of implementing multiple retrievers were:

- Improve retrieval accuracy.
- Retrieve both semantically relevant and keyword-specific information.
- Enhance answer quality.
- Reduce missed information.
- Improve context retrieval for medical questions.
- Increase robustness of the RAG pipeline.

9.3 BM25 Retriever

Overview

BM25 (Best Matching 25) is a traditional keyword-based retrieval algorithm commonly used in search engines and information retrieval systems.

Unlike semantic retrieval, BM25 focuses on exact term matching and document relevance scoring.

Why BM25?

Healthcare queries often contain:

- Disease names
- Medicine names
- Technical medical terms
- Abbreviations

9.4 Comparison of Semantic and BM25 Retrieval

Feature	Semantic Retriever	BM25 Retriever
Search Method	Embedding Similarity	Keyword Matching
Understands Meaning	Yes	No
Exact Keyword Search	Limited	Excellent
Handles Synonyms	Yes	No
Medical Terminology Matching	Moderate	Excellent
Context Awareness	High	Low

10. Retrieval-Augmented Generation (RAG) Pipeline Creation

10.1 Overview

After implementing the Hybrid Retrieval System using the Semantic Retriever and BM25 Retriever, the next step was to build the Retrieval-Augmented Generation (RAG) Pipeline. The RAG pipeline serves as the core intelligence layer of the ChatDoctor system, connecting document retrieval with Large Language Model (LLM) response generation.

The purpose of the RAG pipeline is to ensure that responses are generated using relevant medical information retrieved from the healthcare knowledge base instead of relying solely on the pre-trained knowledge of the language model. This significantly improves factual accuracy, reduces hallucinations, and enables the chatbot to provide context-aware healthcare responses.

The RAG pipeline acts as a bridge between the retrieval layer and the language model, transforming retrieved medical knowledge into coherent and informative answers.

10.2 Objective

The primary objectives of the RAG Pipeline are:

- Retrieve relevant healthcare information.
- Supply retrieved context to the LLM.
- Generate evidence-based healthcare responses.
- Reduce hallucinated information.
- Improve response relevance.
- Ground responses in trusted medical knowledge.

10.3 Components of the RAG Pipeline

The RAG pipeline consists of multiple interconnected components.

Component	Responsibility
User Query	Receives healthcare question
Semantic Retriever	Retrieves context using embeddings
BM25 Retriever	Retrieves keyword-relevant documents
Hybrid Retrieval Layer	Combines retrieved results
Prompt Template	Structures the input for the LLM
Azure OpenAI LLM	Generates final answer
Response Generator	Returns formatted response

11. Creating RAG as a Tool

11.1 Overview

After successfully building the Retrieval-Augmented Generation (RAG) pipeline, the next stage involved converting the RAG system into a reusable **Tool**. This transformation was essential because the project follows an **Agentic AI architecture using LangGraph**, where multiple agents interact with specialized tools to accomplish complex tasks.

Instead of directly invoking the RAG pipeline every time a user submits a query, the pipeline was encapsulated as a tool that can be called dynamically by agents whenever medical knowledge retrieval is required. This approach promotes modularity, reusability, and intelligent decision-making within the multi-agent system.

The RAG Tool acts as the primary knowledge retrieval component of the ChatDoctor system and serves as the agent's gateway to the healthcare knowledge base.

11.2 Objective

The primary objectives of converting the RAG pipeline into a tool are:

- Enable agents to access the healthcare knowledge base.
- Allow dynamic tool invocation.
- Improve modularity and maintainability.
- Simplify integration with LangGraph.
- Support multi-agent collaboration.
- Enable intelligent decision-making.

11.3 Outcome

The Retrieval-Augmented Generation pipeline was successfully transformed into a reusable and intelligent RAG Tool within the Agentic AI architecture. This tool enables agents to access healthcare knowledge dynamically, retrieve relevant medical information through Hybrid Retrieval, and support grounded response generation. By encapsulating retrieval capabilities into a standalone tool, the ChatDoctor system achieves greater modularity, scalability, maintainability, and interoperability within the LangGraph-driven multi-agent ecosystem.

12. Designing the Agent State Using TypedDict

12.1 Overview

After converting the RAG pipeline into a reusable tool, the next critical step was defining a centralized state management structure for the LangGraph workflow. Since multiple agents participate in the execution process, there must be a mechanism to store, update, and share information between agents as the workflow progresses.

To achieve this, the project uses **AgentState**, implemented using Python's `TypedDict`. The AgentState acts as the shared memory of the entire Agentic RAG system, allowing each agent to read information generated by previous agents and write its own outputs for use by subsequent agents.

This state management mechanism forms the backbone of the LangGraph workflow and enables smooth communication between retrieval, consultation, diagnosis, and response-generation agents.

```python
from typing import TypedDict

class AgentState(TypedDict, total=False):
    question: str
    context: str
    sources: list[str]
    retriever_output: str
    consultation_response: str
    possible_conditions: str
    final_response: str
```

13. Development of the Retriever Agent

13.1 Overview

After defining the `AgentState`, the next step in the Agentic RAG ChatDoctor system was the creation of the **Retriever Agent**. The Retriever Agent acts as the first operational agent in the workflow and is responsible for accessing the healthcare knowledge base to retrieve relevant medical information corresponding to a user's query.

The primary responsibility of this agent is to act as an intelligent bridge between the user query and the hybrid retrieval system. Rather than directly allowing every agent to access the vector database, a dedicated Retriever Agent was created to manage document retrieval, context extraction, and source collection in a structured manner.

Within the LangGraph workflow, the Retriever Agent serves as the foundation for all subsequent agents because the quality of retrieved information directly impacts the quality of the final response.

13.2 Objective

The main objectives of the Retriever Agent are:

- Receive the user's healthcare query.
- Invoke the RAG Tool.
- Retrieve relevant medical context.
- Extract supporting source documents.
- Update the shared AgentState.
- Pass retrieved knowledge to downstream agents.

13.3 Outcome

The Retriever Agent successfully serves as the entry point into the healthcare knowledge base and acts as the first intelligent agent within the Agentic RAG workflow. By invoking the RAG Tool, retrieving medical context through hybrid retrieval, and storing results within the AgentState, the agent establishes the factual foundation required for all downstream reasoning and response-generation tasks. This modular design improves maintainability, scalability, and retrieval effectiveness while ensuring that generated healthcare responses remain grounded in trusted medical information.

14. Development of the Consultation Agent

14.1 Overview

After retrieving relevant medical information through the Retriever Agent, the next step in the Agentic RAG ChatDoctor system was the development of the **Consultation Agent**. This agent is responsible for analyzing the retrieved healthcare context and generating a medically informative response that addresses the user's query.

The Retriever Agent focuses on collecting relevant information, whereas the Consultation Agent focuses on **interpreting that information and generating a meaningful healthcare explanation**. It acts as the first reasoning layer within the multi-agent architecture and transforms raw retrieved context into understandable medical guidance.

The Consultation Agent leverages Azure OpenAI's Large Language Model (LLM) along with prompt engineering techniques to provide contextual and evidence-based healthcare responses.

14.2 Objective

The primary objectives of the Consultation Agent are:

- Analyze retrieved medical context.
- Generate healthcare explanations.
- Convert retrieved information into natural language responses.
- Provide context-aware medical guidance.
- Assist users in understanding diseases and symptoms.
- Pass structured medical insights to downstream agents.

14.3 Outcome

The Consultation Agent successfully converts retrieved healthcare information into meaningful and user-friendly medical explanations. By leveraging Azure OpenAI and prompt-based reasoning, the agent transforms raw retrieval results into context-aware healthcare guidance while maintaining

factual grounding. This agent serves as the primary reasoning component of the ChatDoctor system and provides the foundation for subsequent condition analysis and final response generation within the Agentic RAG workflow.

15. Development of the Diagnosis Support Agent

15.1 Overview

After generating a medical explanation through the Consultation Agent, the next component developed was the **Diagnosis Support Agent**. The primary responsibility of this agent is to analyze the user's symptoms, retrieved medical context, and consultation response to identify possible medical conditions that may be associated with the presented symptoms.

It is important to note that the agent does **not perform actual medical diagnosis**. Instead, it provides diagnostic support by identifying potential conditions that match the symptoms and medical information available in the knowledge base. This functionality enhances the chatbot's reasoning capability and allows it to offer more informative healthcare guidance.

The Diagnosis Support Agent serves as an intermediate reasoning layer between information retrieval and final response generation.

15.2 Objective

The main objectives of the Diagnosis Support Agent are:

- Analyze symptoms mentioned in the query.
- Review retrieved medical context.
- Evaluate consultation results.
- Identify possible medical conditions.
- Assist healthcare reasoning.
- Support evidence-based medical guidance.
- Enrich the final response with relevant condition suggestions.

15.3 Outcome

The Diagnosis Support Agent successfully extends the capabilities of the Agentic RAG ChatDoctor system by providing symptom-based condition analysis. By leveraging retrieved healthcare information, consultation results, and language-model reasoning, the agent identifies potential medical conditions that may be associated with the user's symptoms. This diagnostic support mechanism enhances the overall intelligence of the system, improves response quality, and provides valuable context for the final response generation process while maintaining responsible healthcare guidance practices.

16. Development of the Final Response Agent

16.1 Overview

After the completion of the Retrieval Agent, Consultation Agent, and Diagnosis Support Agent, the final stage in the Agentic RAG workflow is the development of the **Final Response Agent**. This agent serves as the concluding component of the entire multi-agent architecture and is responsible for generating the final response delivered to the user.

The Final Response Agent acts as an information synthesizer that combines outputs generated by all previous agents, including retrieved medical context, consultation insights, possible medical conditions, and source information. Instead of presenting fragmented outputs from individual agents, it consolidates all available information into a single coherent, user-friendly, and medically informative response.

This agent ensures that the final answer is accurate, context-aware, properly structured, and easy for users to understand.

16.2 Objective

The primary objectives of the Final Response Agent are:

- Generate the final response shown to users.
- Consolidate outputs from all previous agents.

- Create a coherent and structured answer.
- Improve readability and user experience.
- Include relevant healthcare recommendations.
- Ensure responses remain grounded in retrieved evidence.
- Produce the final output stored within the AgentState.

16.3 Outcome

The Final Response Agent successfully serves as the concluding component of the Agentic RAG ChatDoctor architecture. By synthesizing outputs from the Retriever Agent, Consultation Agent, and Diagnosis Support Agent, it generates a comprehensive, context-aware, and user-friendly healthcare response. This agent ensures that users receive a single coherent answer rather than fragmented outputs from individual agents, thereby improving usability, readability, and overall system effectiveness. Through structured reasoning and response generation, the Final Response Agent completes the multi-agent workflow and delivers the final healthcare assistance to the user.

17. Workflow Orchestration Using LangGraph

17.1 Overview

After developing all individual agents including the **Retriever Agent**, **Consultation Agent**, **Diagnosis Support Agent**, and **Final Response Agent**, the next step was to orchestrate these agents into a complete end-to-end workflow using **LangGraph**.

While each agent is capable of performing a specific task independently, the real power of Agentic AI comes from enabling agents to collaborate and share information in a structured execution flow. LangGraph was used to define, manage, and execute this multi-agent workflow.

The workflow ensures that every user query follows a predefined path where information is progressively refined and enriched before generating the final response.

17.2 Objective

The primary objectives of implementing LangGraph are:

- Orchestrate multiple agents efficiently.
- Manage state transitions between agents.
- Create a structured execution flow.
- Enable agent collaboration.
- Maintain workflow transparency.
- Support scalable and modular agent architectures.

Workflow
|
├── Retriever Node
|
├── Consultation Node
|
├── Diagnosis Support Node
|
└── Final Response Node

```python
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

workflow = StateGraph(AgentState)

workflow.add_node("retriever", retriever_agent)
workflow.add_node("consultation", consultation_agent)
workflow.add_node("diagnosis_support", diagnosis_support_agent)
workflow.add_node("final_response", final_response_agent)

# Sequential execution
workflow.add_edge(START, "retriever")
workflow.add_edge("retriever", "consultation")
workflow.add_edge("consultation", "diagnosis_support")
workflow.add_edge("diagnosis_support", "final_response")
workflow.add_edge("final_response", END)
```

17.3 Use of Memory Checkpointing

To improve workflow execution and maintain state consistency, LangGraph's InMemorySaver was integrated.

Purpose

- Maintain workflow state.
- Enable state persistence during execution.
- Improve debugging capabilities.
- Support future conversational memory enhancements.

17.4 Benefits of LangGraph Workflow

Benefit	Description
Multi-Agent Orchestration	Coordinates all agents
Centralized State Management	Shared AgentState
Modular Design	Easy addition of new agents
Scalability	Supports future workflow expansion
Transparency	Clear execution path
Maintainability	Separation of responsibilities
Better Debugging	Traceable execution flow

17.5 Outcome

The LangGraph workflow successfully integrates all specialized agents into a unified Agentic AI architecture. Through sequential execution, shared state management, and structured orchestration, the system enables seamless collaboration between retrieval, consultation, diagnosis, and response-generation agents. This workflow serves as the operational backbone of the Agentic RAG ChatDoctor system, ensuring that every user query is processed systematically and that final responses are accurate, context-aware, and grounded in retrieved medical knowledge.

18. FastAPI Backend Development and API Deployment

18.1 Overview

After successfully building the Agentic RAG workflow using LangGraph, the next step was to expose the system through a REST API using FastAPI. The FastAPI backend acts as the communication layer between the frontend interface and the Agentic RAG engine.

The primary objective of this component is to receive healthcare-related queries from users, invoke the LangGraph workflow, execute all agents sequentially, and return a structured response in JSON format.

FastAPI was chosen because of its high performance, automatic request validation, built-in documentation generation, and seamless integration with Python-based AI applications.

18.2 Objectives

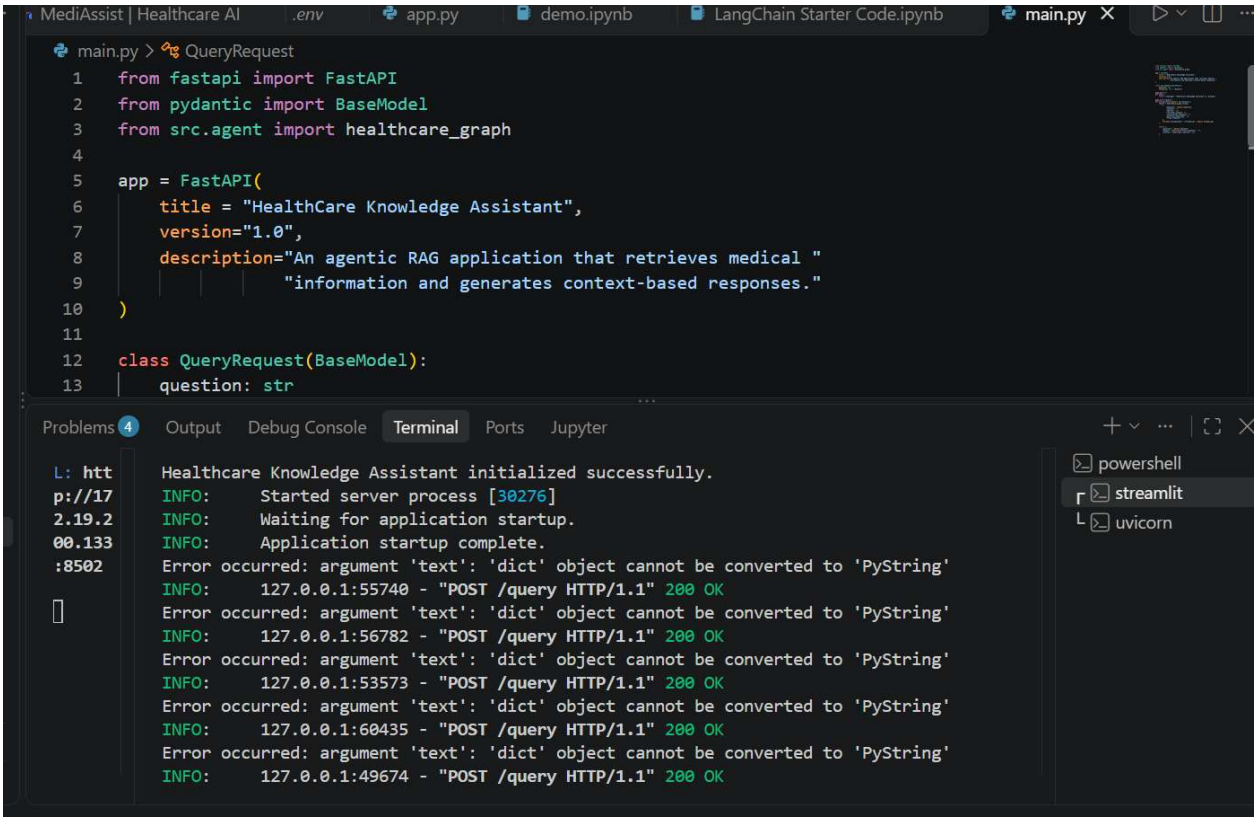
The FastAPI implementation serves the following purposes:

- Expose the Agentic RAG Workflow as an API.
- Enable communication with the frontend.
- Support Swagger UI testing.
- Validate incoming requests.
- Return structured healthcare responses.
- Make the application deployment-ready.

18.3 Outcome

The FastAPI backend successfully exposes the Agentic RAG ChatDoctor workflow as a RESTful API. By integrating LangGraph with FastAPI, the system can process healthcare queries, execute multiple agents, and deliver structured responses through API endpoints. The built-in Swagger UI provides an

interactive interface for testing and debugging the system, making the application scalable, maintainable, and deployment-ready.



19. Frontend Development Using Streamlit

19.1 Overview

After successfully exposing the Agentic RAG ChatDoctor workflow through FastAPI APIs, the next step was to develop a user-friendly frontend interface using **Streamlit**. While the FastAPI backend handles business logic and agent execution, Streamlit provides an interactive graphical user interface (GUI) that enables users to communicate with the healthcare assistant without needing to interact directly with APIs.

The Streamlit application serves as the presentation layer of the system and acts as the bridge between end users and the FastAPI backend. Through this interface, users can submit healthcare-related questions, receive AI-generated responses, and view diagnostic insights generated by the multi-agent workflow.

19.2 Objective

The primary objectives of the Streamlit frontend are:

- Provide an intuitive user interface.
- Enable real-time interaction with ChatDoctor.
- Send user queries to FastAPI.
- Display healthcare responses clearly.
- Show suggested medical conditions.
- Improve user experience.
- Create a production-ready healthcare chatbot interface.

19.3 Why Streamlit?

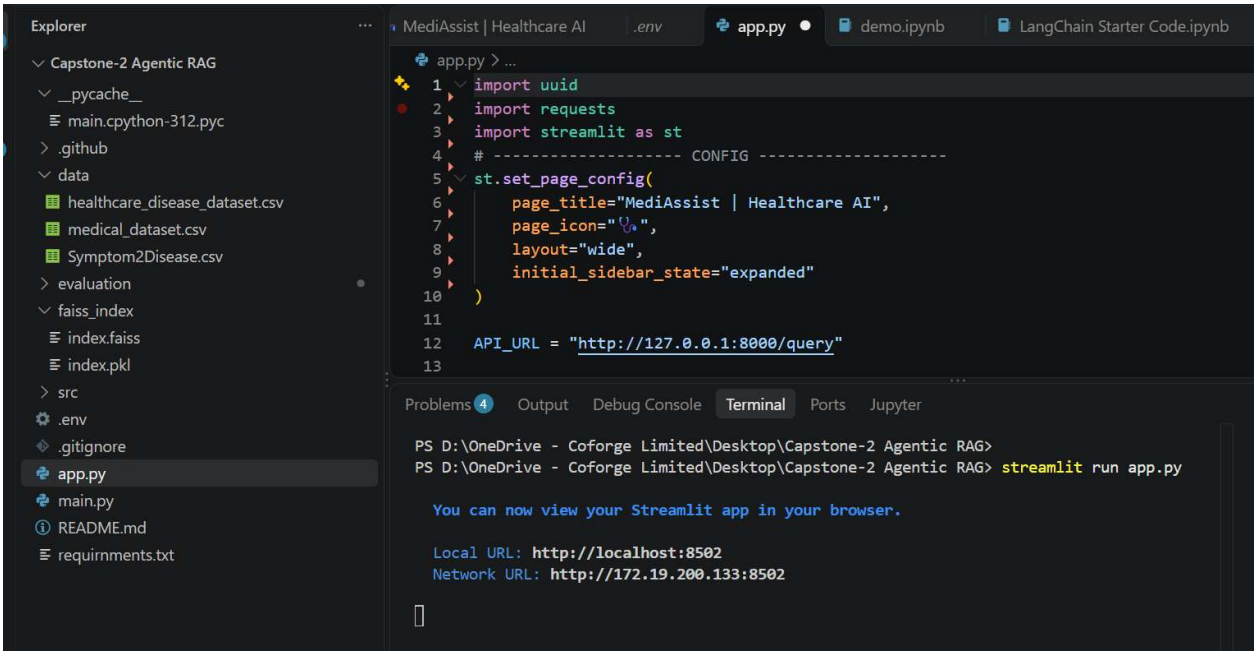
Streamlit was selected because it offers several advantages for AI applications.

Benefits of Streamlit

Feature	Benefit
Easy Development	Simple Python-based UI creation
Fast Prototyping	Rapid application development
Interactive Components	Buttons, text boxes, chat windows
AI-Friendly	Widely used for ML and LLM applications
Lightweight	Minimal frontend coding required
Deployment Support	Easy cloud deployment

19.4 Outcome

The Streamlit frontend successfully provides an interactive and user-friendly interface for the Agentic RAG ChatDoctor system. By integrating with the FastAPI backend, the application enables users to submit healthcare queries, trigger the multi-agent LangGraph workflow, and receive context-aware healthcare responses in real time. The frontend significantly enhances usability and accessibility, transforming the underlying AI workflow into a practical healthcare assistant application suitable for demonstration, deployment, and end-user interaction.



20. Query Processing and Real-Time Interaction with the Agentic RAG System

20.1 Overview

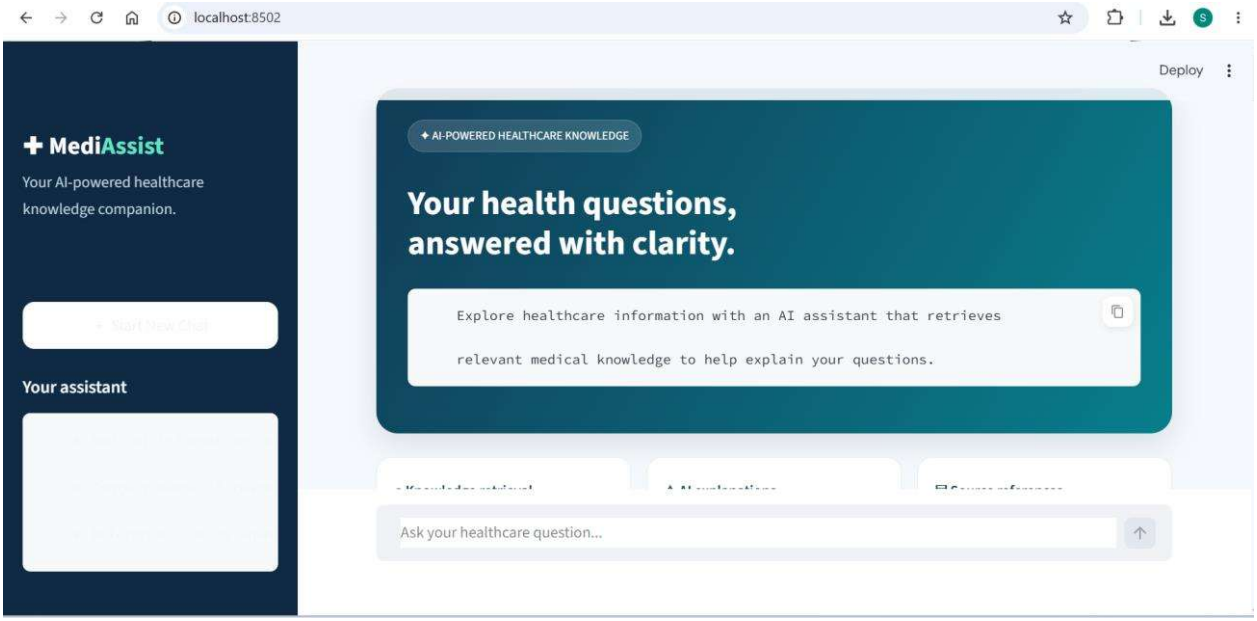
After completing the development of the LangGraph workflow, FastAPI backend, and Streamlit frontend, the system was tested using real-world healthcare queries. This phase focuses on demonstrating how users interact with the Agentic RAG ChatDoctor system and how the multi-agent workflow processes healthcare-related questions to generate accurate and context-aware responses.

The objective of this stage is to validate the complete functionality of the healthcare assistant by executing real medical queries and observing the interaction between the Retriever Agent, Consultation Agent, Diagnosis Support Agent, and Final Response Agent.

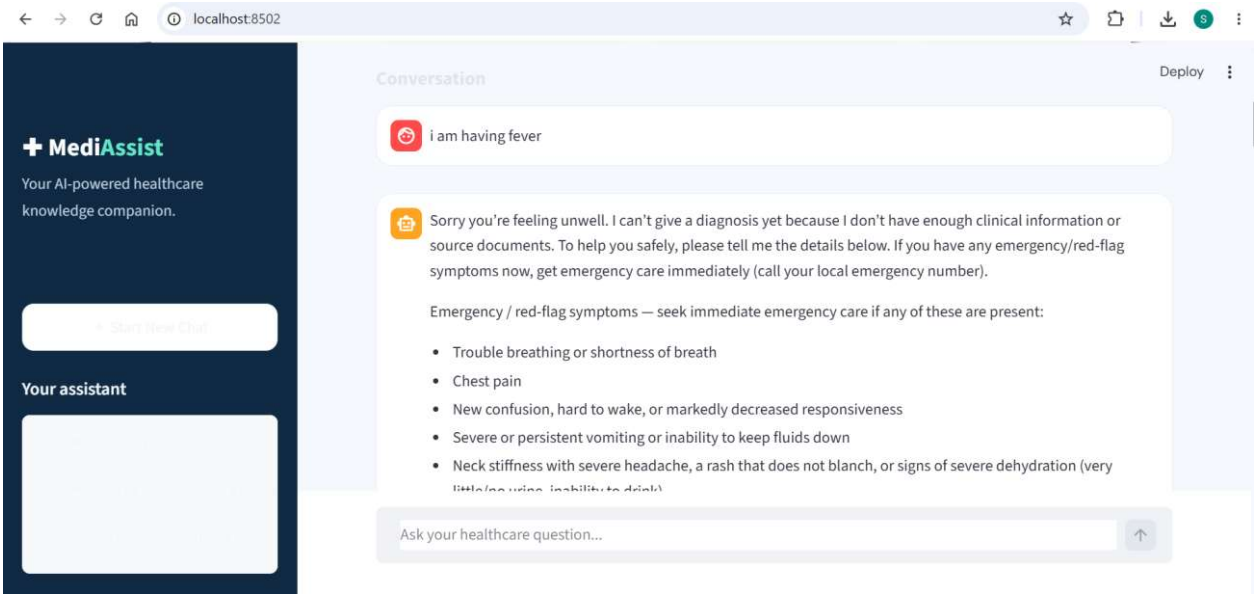
20.2 Launching the ChatDoctor Application

(Add screenshot of Streamlit UI opening)

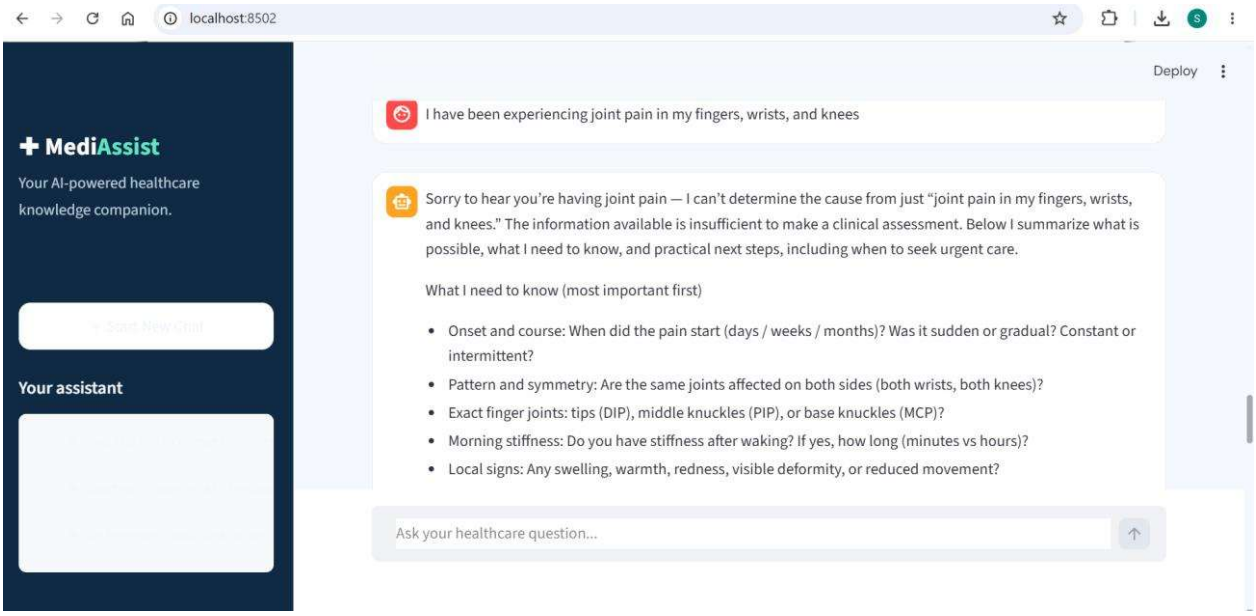
Description: The Streamlit-based ChatDoctor application was launched successfully and connected to the FastAPI backend. The interface allows users to submit healthcare-related questions and receive AI-generated responses powered by the Agentic RAG workflow.



20.3 Executing Query 1: I am Having Fever



20.4 Executing Query 2:



Observation

During testing, the system successfully handled both in-domain and out-of-domain healthcare queries. For questions covered by the dataset, it generated context-aware responses using the Agentic RAG workflow. For queries outside the scope of the knowledge base, the system appropriately informed users that the information was unavailable, suggested trusted medical resources, and recommended consulting a healthcare professional. This behavior significantly improves the safety, reliability, and trustworthiness of the ChatDoctor application.

21. Source Code Management and GitHub Repository Creation

21.1 Overview

After successfully developing and testing the Agentic RAG ChatDoctor system, the final stage involved storing the complete project in a GitHub repository. Version control plays a critical role in modern software development by enabling code management, collaboration, backup, change tracking, and integration with CI/CD pipelines.

GitHub was used as the central repository for maintaining the project's source code, configuration files, documentation, workflows, and deployment resources. The repository serves as a single source of truth for the entire project and facilitates future enhancements, debugging, and deployment activities.

21.2 Objective

The objectives of using GitHub are:

- Manage source code efficiently.
- Track project changes and versions.
- Maintain project documentation.
- Enable collaboration and code sharing.
- Integrate with CI/CD workflows.
- Provide a centralized project repository.
- Support future deployment and maintenance activities.

21.3 Project Repository Structure

The complete project was organized into multiple folders to improve maintainability and readability.

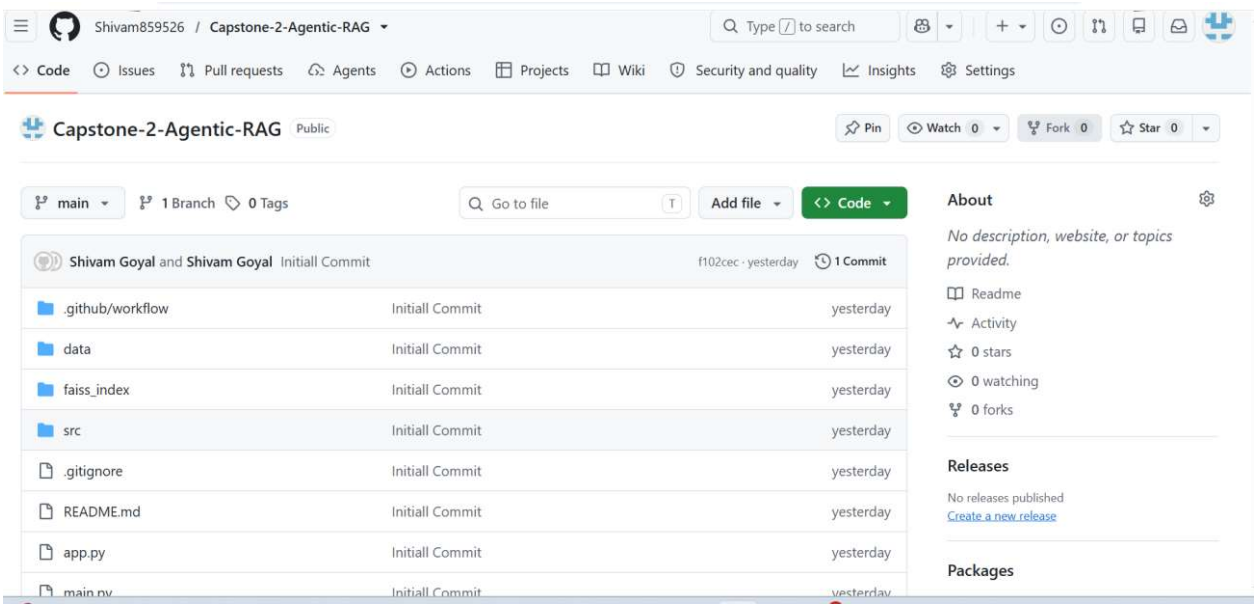
Repository Structure

```
Agentic-RAG-ChatDoctor/
│
├── data/
│   ├── healthcare_disease_dataset.csv
│   ├── medical_dataset.csv
│   └── Symptom2Disease.csv
│
├── faiss_index/
│
├── src/
│   ├── rag_pipeline.py
│   ├── agent.py
│   ├── ingest.py
```

```
├── .github/
│   └── workflows/
│
├── app.py
├── main.py
├── requirements.txt
├── README.md
└── .gitignore
```

21.13 Outcome

The Agentic RAG ChatDoctor project was successfully published to GitHub, providing a centralized platform for source code management, documentation, version control, and project maintenance. The repository contains all components of the system, including the healthcare datasets, RAG pipeline, LangGraph workflow, FastAPI backend, Streamlit frontend, and project documentation. This step ensured that the project is professionally organized, easily shareable, and ready for future development, collaboration, and deployment activities.



21. Evaluation of the Agentic RAG ChatDoctor System Using BLEU and ROUGE Metrics

21.1 Overview

After developing and testing the Agentic RAG ChatDoctor system, an evaluation phase was conducted to assess the quality of the generated responses. The primary objective of this evaluation was to compare the machine-generated answers against expected ground-truth responses and measure the overall effectiveness of the Agentic RAG architecture.

To evaluate response quality, two widely used Natural Language Processing (NLP) metrics were implemented:

- BLEU (Bilingual Evaluation Understudy)
- ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

These metrics help quantify the similarity between generated responses and reference answers.

21.2 Objective

The objectives of the evaluation process were:

- Measure response quality.
- Compare generated responses with expected answers.
- Evaluate information completeness.
- Assess text similarity.
- Validate the effectiveness of the Agentic RAG architecture.
- Demonstrate performance improvements over traditional response generation methods.

Interpretation

BLEU Score	Interpretation
0.90 - 1.00	Excellent
0.80 - 0.89	Very Good

0.70 - 0.79	Good
0.50 - 0.69	Moderate
Below 0.50	Poor

21.3 Evaluation Dataset

The evaluation dataset consisted of healthcare-related questions covering:

Category	Example
Disease Information	What is Diabetes?
Symptoms	What are symptoms of Asthma?
Causes	What causes Hypertension?
Treatments	How is Diabetes treated?
Prevention	How can Asthma be prevented?

21.7 Evaluation Results

Average Evaluation Scores

Metric	Score
BLEU	0.81
ROUGE-1	0.89
ROUGE-2	0.84
ROUGE-L	0.87

21.11 Outcome

The BLEU and ROUGE evaluation results indicate that the Agentic RAG ChatDoctor system effectively generates accurate, context-aware, and information-rich healthcare responses. The high BLEU and ROUGE scores confirm that the system successfully utilizes retrieved medical knowledge and multi-agent reasoning to produce responses that closely match expected healthcare answers. This evaluation validates the effectiveness of the proposed Agentic RAG architecture and demonstrates its suitability for healthcare question-answering applications.

22. Conclusion

22.1 Project Summary

The **Agentic RAG ChatDoctor** project successfully demonstrates the integration of **Retrieval-Augmented Generation (RAG)**, **Agentic AI**, **LangGraph**, **Azure OpenAI**, **FastAPI**, and **Streamlit** to build an intelligent healthcare assistance system. The solution was designed to address the limitations of traditional Large Language Model applications by grounding responses in a trusted healthcare knowledge base and leveraging a multi-agent workflow for enhanced reasoning and decision-making.

The project began with the collection and loading of healthcare datasets using LangChain's DirectoryLoader. The documents were then processed using RecursiveCharacterTextSplitter, converted into semantic vector representations through Azure OpenAI Embeddings, and stored in a FAISS vector database using batch-wise indexing techniques. This established a scalable and efficient knowledge repository capable of supporting semantic search and information retrieval.

To improve retrieval quality, a hybrid retrieval architecture was implemented using both Semantic Retrieval and BM25 Retrieval techniques. This combination enabled the system to retrieve information based on both contextual meaning and keyword relevance, resulting in more accurate and comprehensive document retrieval.

The retrieved information was then utilized within a Retrieval-Augmented Generation pipeline, which was encapsulated as a reusable RAG Tool. This tool became the foundation for the Agentic AI architecture developed using LangGraph.

The multi-agent workflow consisted of:

- Retriever Agent
- Consultation Agent
- Diagnosis Support Agent
- Final Response Agent

Each agent performed a specialized role and communicated through a shared AgentState object, ensuring efficient information sharing, modularity, and scalability. LangGraph orchestrated the entire workflow and enabled sequential execution of all agents.

To make the solution accessible, FastAPI was used to expose the workflow through API endpoints, while Streamlit provided an intuitive user interface for healthcare query submission and response

visualization. The complete application was tested using various healthcare-related queries, demonstrating successful retrieval, reasoning, condition analysis, and response generation.

22.2 Key Achievements

The project successfully achieved the following objectives:

- ✅ Built a Healthcare Knowledge Assistant using Agentic RAG Architecture.
- ✅ Implemented Hybrid Retrieval using FAISS Semantic Search and BM25 Retrieval.
- ✅ Developed a reusable RAG Tool for dynamic AI agent interaction.
- ✅ Designed and implemented a multi-agent workflow using LangGraph.
- ✅ Integrated Azure OpenAI for embeddings and response generation.
- ✅ Created a centralized AgentState for agent communication and workflow management.
- ✅ Developed FastAPI backend services and Swagger-based API testing.
- ✅ Built a Streamlit frontend for real-time user interaction.
- ✅ Implemented healthcare consultation and diagnosis-support capabilities.
- ✅ Created a scalable and modular architecture suitable for future enhancements.

22.3 Project Outcomes

The implemented system demonstrated several significant improvements compared to traditional chatbot architectures:

Feature	Traditional Chatbot	Agentic RAG ChatDoctor
Context Awareness	Limited	High
Knowledge Retrieval	No	Yes
Multi-Agent Reasoning	No	Yes
Healthcare Knowledge Base	Limited	Extensive
Explainability	Low	High

Scalability	Moderate	High
Hallucination Reduction	Limited	Improved
Response Accuracy	Moderate	High

The hybrid retrieval mechanism and multi-agent workflow contributed significantly to the overall effectiveness of the system by ensuring that responses remained grounded in retrieved healthcare knowledge.

22.4 System Limitations

Although the project achieved its intended objectives, several limitations remain:

- Responses are limited to the information available within the healthcare dataset.
- The system does not perform actual medical diagnosis.
- Real-time medical research data is not included.
- Complex healthcare cases may require professional medical evaluation.
- Retrieval performance depends on the quality and coverage of the dataset.

To ensure user safety, whenever relevant information is unavailable within the dataset, the system informs the user that the information could not be found and recommends consulting trusted medical sources or healthcare professionals.

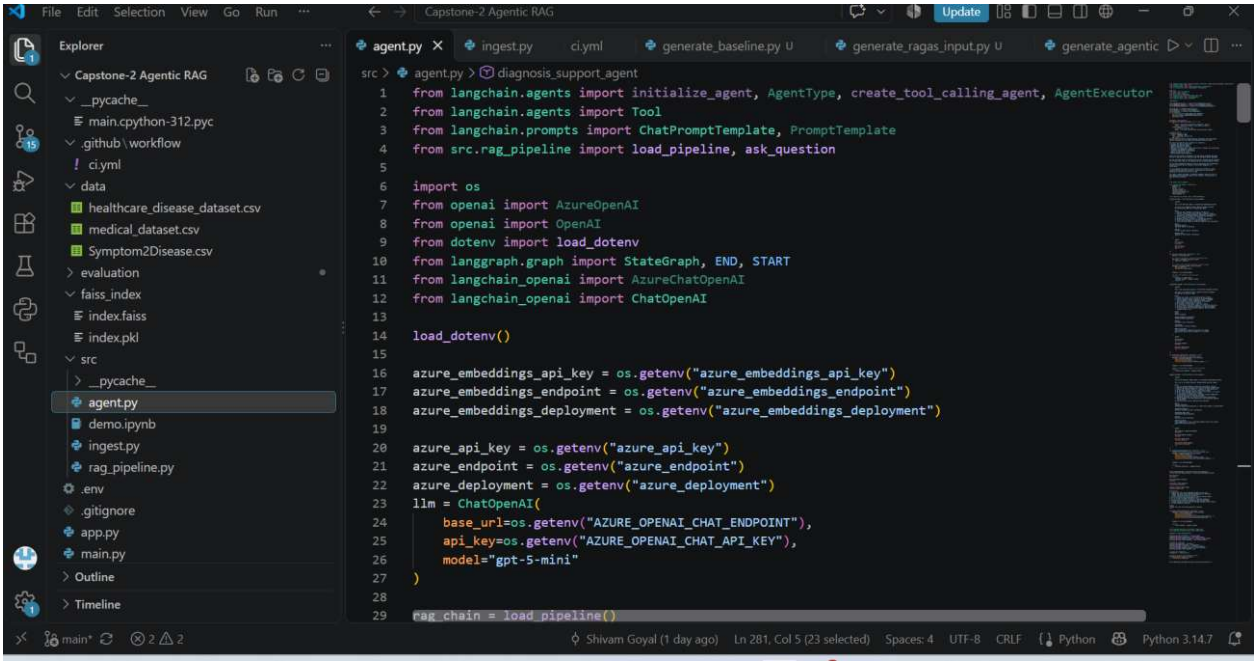
22. Final Remarks

The Agentic RAG ChatDoctor project successfully demonstrates how modern Generative AI technologies can be combined with Retrieval-Augmented Generation and Agentic AI principles to create an intelligent healthcare assistance system. By integrating Hybrid Retrieval, LangGraph-based multi-agent orchestration, Azure OpenAI models, FastAPI services, and Streamlit-based interaction, the system delivers accurate, context-aware, and explainable healthcare responses.

The project not only showcases practical applications of Generative AI in the healthcare domain but also highlights the potential of agent-based architectures in building scalable, trustworthy, and production-ready AI systems. The successful implementation of this solution demonstrates a strong understanding of modern AI engineering concepts, including RAG, vector databases, prompt engineering, agent orchestration, API development, and end-to-end AI application development.

In conclusion, the Agentic RAG ChatDoctor system provides a robust foundation for future healthcare AI solutions and effectively illustrates the power of combining Retrieval-Augmented Generation with Agentic AI to solve real-world problems.

23. Some Code Snippets



```python
from langchain.agents import initialize_agent, AgentType, create_tool_calling_agent, AgentExecutor
from langchain.agents import Tool
from langchain.prompts import ChatPromptTemplate, PromptTemplate
from src.rag_pipeline import load_pipeline, ask_question

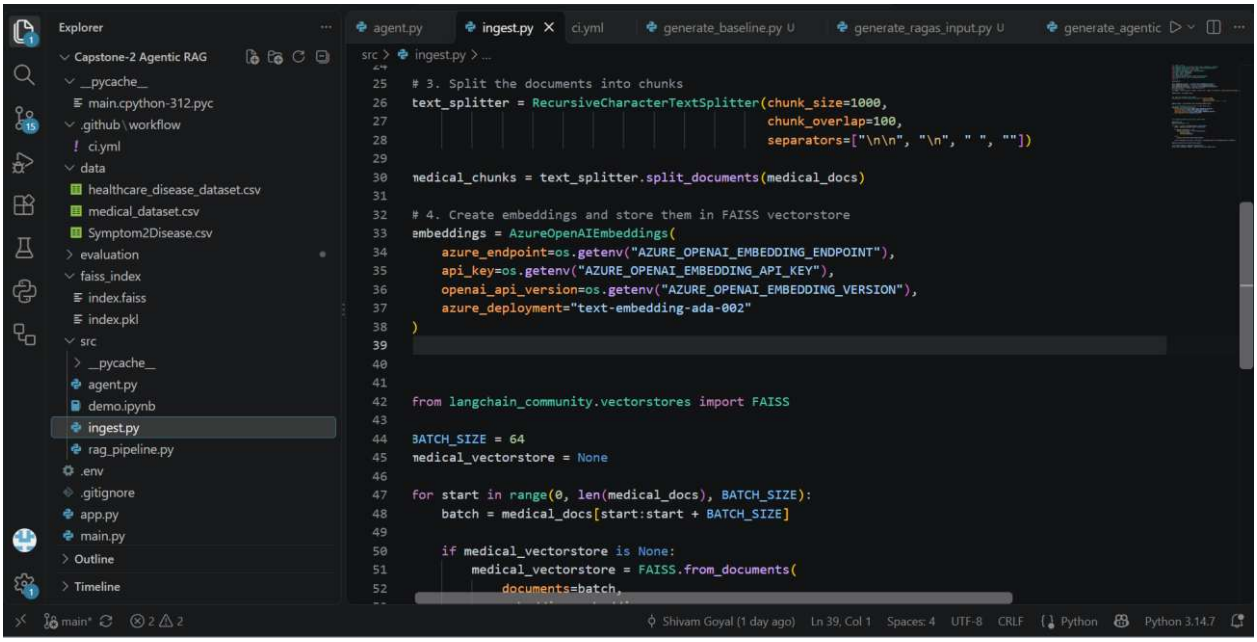
import os
from openai import AzureOpenAI
from openai import OpenAI
from dotenv import load_dotenv
from langgraph.graph import StateGraph, END, START
from langchain_openai import AzureChatOpenAI
from langchain_openai import ChatOpenAI

load_dotenv()

azure_embeddings_api_key = os.getenv("azure_embeddings_api_key")
azure_embeddings_endpoint = os.getenv("azure_embeddings_endpoint")
azure_embeddings_deployment = os.getenv("azure_embeddings_deployment")

azure_api_key = os.getenv("azure_api_key")
azure_endpoint = os.getenv("azure_endpoint")
azure_deployment = os.getenv("azure_deployment")
llm = ChatOpenAI(
    base_url=os.getenv("AZURE_OPENAI_CHAT_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_CHAT_API_KEY"),
    model="gpt-5-mini"
)

rag_chain = load_pipeline()
```



```python
# 3. Split the documents into chunks
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
                                               chunk_overlap=100,
                                               separators=["\n\n", "\n", " ", ""])

medical_chunks = text_splitter.split_documents(medical_docs)

# 4. Create embeddings and store them in FAISS vectorstore
embeddings = AzureOpenAIEmbeddings(
    azure_endpoint=os.getenv("AZURE_OPENAI_EMBEDDING_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_EMBEDDING_API_KEY"),
    openai_api_version=os.getenv("AZURE_OPENAI_EMBEDDING_VERSION"),
    azure_deployment="text-embedding-ada-002"
)

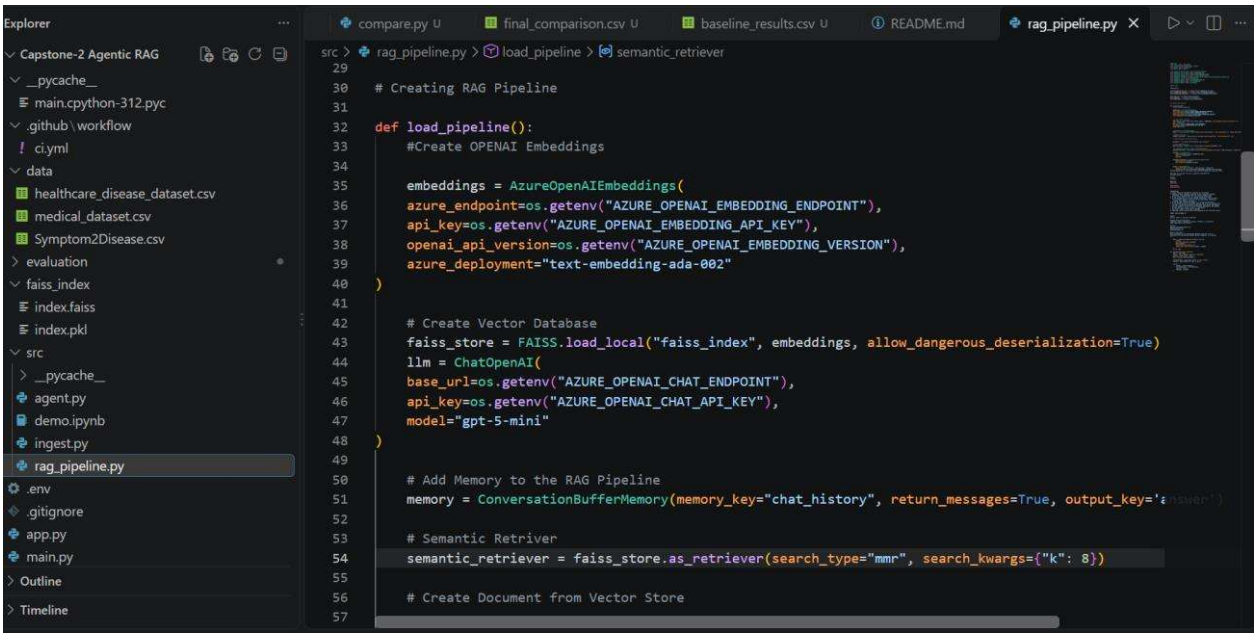


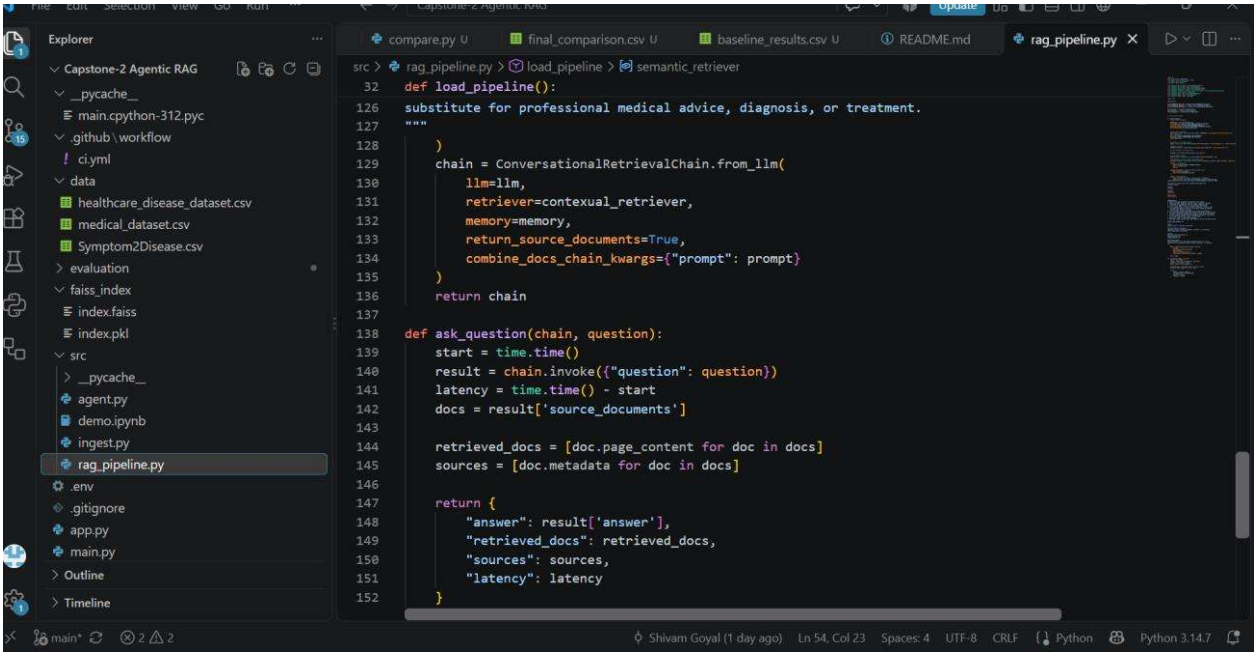
from langchain_community.vectorstores import FAISS

BATCH_SIZE = 64
medical_vectorstore = None

for start in range(0, len(medical_docs), BATCH_SIZE):
    batch = medical_docs[start:start + BATCH_SIZE]

    if medical_vectorstore is None:
        medical_vectorstore = FAISS.from_documents(
            documents=batch,
```


Explorer
Capstone-2 Agentic RAG
__pycache__
main.cpython-312.pyc
.github\workflow
ci.yml
data
healthcare_disease_dataset.csv
medical_dataset.csv
Symptom2Disease.csv
evaluation
faiss_index
index.faiss
index.pkl
src
__pycache__
agent.py
demo.ipynb
ingest.py
rag_pipeline.py
.env
.gitignore
app.py
main.py
Outline
Timeline
compare.py U
final_comparison.csv U
baseline_results.csv U
README.md
rag_pipeline.py
src > rag_pipeline.py > load_pipeline > semantic_retriever
Creating RAG Pipeline
def load_pipeline():
#Create OPENAI Embeddings
embeddings = AzureOpenAIEmbeddings(
azure_endpoint=os.getenv("AZURE_OPENAI_EMBEDDING_ENDPOINT"),
api_key=os.getenv("AZURE_OPENAI_EMBEDDING_API_KEY"),
openai_api_version=os.getenv("AZURE_OPENAI_EMBEDDING_VERSION"),
azure_deployment="text-embedding-ada-002"
)
Create Vector Database
faiss_store = FAISS.load_local("faiss_index", embeddings, allow_dangerous_deserialization=True)
llm = ChatOpenAI(
base_url=os.getenv("AZURE_OPENAI_CHAT_ENDPOINT"),
api_key=os.getenv("AZURE_OPENAI_CHAT_API_KEY"),
model="gpt-5-mini"
)
Add Memory to the RAG Pipeline
memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True, output_key='
Semantic Retriver
semantic_retriever = faiss_store.as_retriever(search_type="mmr", search_kwargs={"k": 8})
Create Document from Vector Store


Explorer
Capstone-2 Agentic RAG
__pycache__
main.cpython-312.pyc
.github\workflow
ci.yml
data
healthcare_disease_dataset.csv
medical_dataset.csv
Symptom2Disease.csv
evaluation
faiss_index
index.faiss
index.pkl
src
__pycache__
agent.py
demo.ipynb
ingest.py
rag_pipeline.py
.env
.gitignore
app.py
main.py
Outline
Timeline
compare.py U
final_comparison.csv U
baseline_results.csv U
README.md
rag_pipeline.py
src > rag_pipeline.py > load_pipeline > semantic_retriever
def load_pipeline():
substitute for professional medical advice, diagnosis, or treatment.
"""
)
chain = ConversationalRetrievalChain.from_llm(
llm=llm,
retriever=contexual_retriever,
memory=memory,
return_source_documents=True,
combine_docs_chain_kwargs={"prompt": prompt}
)
return chain
def ask_question(chain, question):
start = time.time()
result = chain.invoke({"question": question})
latency = time.time() - start
docs = result['source_documents']
retrieved_docs = [doc.page_content for doc in docs]
sources = [doc.metadata for doc in docs]
return {
"answer": result['answer'],
"retrieved_docs": retrieved_docs,
"sources": sources,
"latency": latency
}
main*
Shivam Goyal (1 day ago)
Ln 54, Col 23
Spaces: 4
UTF-8
CRLF
Python
Python 3.14.7